

ASDD PRACTICAL VIVA QUESTIONS

FINAL INDUSTRY-READY EDITION

Six experiments • Clear answers • Workflow graphics

01 JIRA

02 GIT

03 CI PIPELINE

04 DOCKER

05 COMPOSE

06 KUBERNETES

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

43-PAGE SOURCE PDF

How to use this guide

Prepared from the uploaded **ASDD PRACTICAL EXP (1).pdf** (43 pages). The guide follows all six experiments in source order and emphasizes definitions, procedures, commands and expected outputs useful for an oral viva.

End-to-end DevOps flow



Workflow overview

Six experiments at a glance

NO.	EXPERIMENT	HIGH-VALUE TOPICS
01	Agile Software Project Lifecycle using Jira	SDLC, Agile, Jira, Scrum, backlog, DevOps
02	Version Control Operations using Git	Repository, staging, commits, branches, merging
03	Continuous Integration using GitHub Actions or Jenkins	Workflow, YAML, triggers, runner, Actions tab
04	Containerizing Applications using Docker	Dockerfile, image, container, build/run
05	Multi-Service Application using Docker Compose	Nginx, Redis, ports, Compose commands
06	Deploying and Scaling Applications using Kubernetes	Cluster, Pod, Deployment, replicas, scaling

Strong viva answer formula

1 · DEFINE	2 · PURPOSE	3 · PRACTICAL LINK
Give a simple meaning.	Say why it is used.	Mention the command, tool or output.

Note: Answers are concise revision explanations based on the practical PDF. Use the actual project, branch and Deployment names from your own setup when demonstrating commands.

EXPERIMENT 01

Agile Software Project Lifecycle using Jira

Source PDF pages Pages 1–9 • 20 viva questions

Agile SDLC cycle



Workflow overview

Q01. What is SDLC?
Answer: Software Development Life Cycle: a systematic process to plan, develop, test, deploy and maintain software.

Q02. What is Agile SDLC?
Answer: An iterative and incremental approach emphasizing flexibility, collaboration and customer feedback.

Q03. Name the four major Agile factors in the PDF.
Answer: Early customer involvement, iterative development, self-organizing teams and adaptation to change.

Q04. Agile vs Waterfall?
Answer: Agile delivers in iterations and adapts to change; Waterfall generally follows sequential phases.

Q05. What are the Agile SDLC steps in the PDF?
Answer: Requirement gathering, design, coding, testing/QA, deployment and feedback.

Q06. What happens during requirement gathering?
Answer: The team works with stakeholders to understand needs and prioritize requirements.

Q07. What is a user story?
Answer: A short feature description from the user's point of view, often written as: As a user, I want… so that…

Q08. What is Jira?
Answer: A project and issue-tracking tool used to plan, assign, track and manage work.

Q09. What is Scrum?
Answer: An Agile framework that organizes work into short iterations called Sprints.

Q10. What is a Sprint?
Answer: A fixed time period in which a team aims to complete selected work items.

Q11. What is a Product Backlog?
Answer: A prioritized list of product features, requirements, fixes and other work.

Q12. What is an Epic?
Answer: A larger body of work that groups related stories and tasks.

Q13. What is a Scrum board?
Answer: A visual board showing tasks by status, such as To Do, In Progress and In Review.

Q14. What is a Sprint goal?
Answer: A short statement of the main outcome the team aims to achieve in the Sprint.

Q15. What is DevOps?
Answer: Practices that bring development and operations together through collaboration, automation and continuous improvement.

Q16. Name the DevOps stages from the PDF.
Answer: Plan, Code, Build, Test, Release, Deploy, Operate and Monitor.

Q17. Give examples of Plan and Code tools.
Answer: Jira can support planning; Git and GitHub support source-code version control.

Q18. Why is feedback important in Agile?
Answer: It helps refine requirements, improve quality and respond to user needs.

Q19. What did you do in the Jira practical?
Answer: Selected a Scrum template, configured a project, viewed the backlog, created an Epic, placed work in a Sprint and viewed the Scrum board.

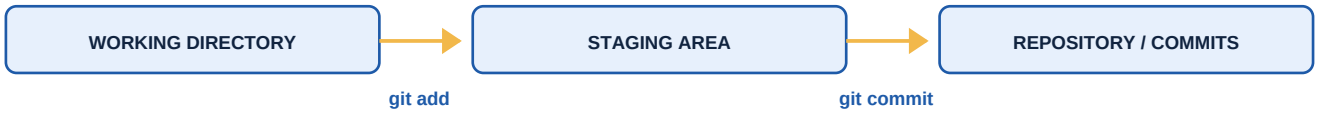
Q20. What is the aim of Experiment 1?
Answer: To study and implement the Agile software project lifecycle using Jira with DevOps pipeline integration.

EXPERIMENT 02

Version Control Operations using Git

Source PDF pages Pages 10–15 • 22 viva questions

Git change flow



Q01. What is version control?
Answer: A system that tracks file changes over time and lets developers inspect or restore earlier versions.

Q02. What is Git?
Answer: A distributed version control system for tracking source-code changes and collaborative development.

Q03. What is a Git repository?
Answer: A project managed by Git that stores files and their version history.

Q04. What does git init do?
Answer: Initializes a Git repository in the current directory.

Q05. Working directory vs staging area vs repository?
Answer: Working directory holds current files; staging selects changes for the next commit; repository stores commits.

Q06. Explain the basic Git workflow.
Answer: Working Directory → Staging Area → Repository. git add stages changes; git commit records them.

Q07. What does git --version do?
Answer: Displays the installed Git version.

Q08. Why use git config?
Answer: To configure settings such as the author name and email recorded with commits.

Q09. What does git status show?
Answer: The current branch and whether files are modified, staged or untracked.

Q10. What does git add . do?
Answer: Stages changes in the current directory and its subdirectories.

Q11. What is a commit?
Answer: A saved snapshot of staged changes with a unique identifier and message.

Q12. What does git commit -m "Initial commit" do?
Answer: Records staged changes with the message Initial commit.

Q13. What does git log --oneline do?
Answer: Shows commit history in a compact one-line format.

Q14. What is a branch?
Answer: An independent line of development used to work on a feature separately.

Q15. How do you create a feature branch?
Answer: git branch feature

Q16. How do you switch to a branch?
Answer: git checkout feature. Modern alternative: git switch feature.

Q17. What is merging?
Answer: Combining changes from one branch into another.

Q18. How do you merge feature into main?
Answer: git checkout main, then git merge feature. Resolve conflicts if Git reports them.

Q19. What is a merge conflict?
Answer: A situation where Git cannot automatically reconcile competing changes.

Q20. Git vs GitHub?
Answer: Git is version-control software; GitHub is an online platform for hosting repositories and collaboration.

Q21. Why is Git important in DevOps?
Answer: It keeps change history, supports collaboration and integrates with CI/CD.

Q22. What is the result of Experiment 2?
Answer: Git configuration, repository initialization, status, staging, commits, history, branching, switching and merging were practised.

EXPERIMENT 03

Continuous Integration using GitHub Actions or Jenkins

Source PDF pages Pages 16–21 • 21 viva questions

CI pipeline flow



Workflow overview

Q01. What is Continuous Integration (CI)?
Answer: Frequently integrating code into a shared repository and running automated checks to detect issues early.

Q02. What is a CI pipeline?
Answer: An automated sequence that checks, builds or tests code after a configured event.

Q03. What are CI benefits?
Answer: Early bug detection, automated checks, improved code quality, time savings and collaboration.

Q04. What tools are required?
Answer: Git, GitHub account, Visual Studio Code and GitHub Actions or Jenkins.

Q05. What is GitHub Actions?
Answer: A GitHub feature that automates workflows such as testing, building and deployment.

Q06. What is Jenkins?
Answer: An automation server used to automate build, test and deployment tasks.

Q07. Why does the PDF recommend GitHub Actions for beginners?
Answer: It is built into GitHub and avoids installing a separate Jenkins server for the basic workflow.

Q08. What is a workflow?
Answer: An automated process defined in a YAML file with triggers, jobs and steps.

Q09. Where are workflow files stored?
Answer: In .github/workflows, for example .github/workflows/ci.yml.

Q10. What is YAML?
Answer: A human-readable data format used for configuration.

Q11. What is a runner?
Answer: A machine or execution environment that runs a workflow job, such as Ubuntu.

Q12. What does actions/checkout do?
Answer: Checks out repository code so later workflow steps can access the files.

Q13. What sample file is made in the practical?
Answer: hello.txt containing the text Hello Continuous Integration.

Q14. What triggers the workflow in the PDF?
Answer: A push to the main branch, according to the workflow configuration.

Q15. What does git clone do?
Answer: Creates a local copy of a remote repository.

Q16. What does git push origin main do?
Answer: Pushes the local main branch to the remote repository named origin.

Q17. How do you commit the workflow?
Answer: git add . followed by git commit -m "Added CI workflow".

Q18. How do you verify pipeline execution?
Answer: Open the repository on GitHub and select the Actions tab.

Q19. What does a green check mark mean?
Answer: The workflow completed successfully.

Q20. CI vs CD?
Answer: CI integrates and validates code; CD automates delivery or deployment of validated changes.

Q21. What is the result of Experiment 3?
Answer: A workflow was configured to run automatically after code was pushed to GitHub.

EXPERIMENT 04

Containerizing Applications using Docker

Source PDF pages Pages 22–26 • 21 viva questions

Docker build and run flow



Workflow overview

Q01. What is Docker?
Answer: A platform for packaging and running applications in containers.

Q02. What is containerization?
Answer: Packaging an application and its dependencies into an isolated, reproducible environment.

Q03. What is a container?
Answer: An isolated environment in which an application and its dependencies run.

Q04. What is a Docker image?
Answer: A template used to create containers.

Q05. What is a Dockerfile?
Answer: A text file containing instructions used to build a Docker image.

Q06. Which application is containerized?
Answer: A simple Python app.py program that prints Hello from Docker!.

Q07. What software is required?
Answer: Windows 10/11, VS Code, Docker Desktop, Python and a web browser are listed in the PDF.

Q08. What does FROM python:3.11 mean?
Answer: Selects the Python 3.11 image as the base image.

Q09. What does WORKDIR /app do?
Answer: Sets /app as the working directory inside the image.

Q10. What does COPY app.py . do?
Answer: Copies app.py into the current working directory in the image.

Q11. What does CMD ["python", "app.py"] do?
Answer: Sets the default command to run the Python app when the container starts.

Q12. How do you check Docker installation?
Answer: docker --version

Q13. What does docker build -t myapp . do?
Answer: Builds an image named myapp using the Dockerfile in the current directory.

Q14. What does the dot in the build command mean?
Answer: The current directory is the build context.

Q15. What does docker images do?
Answer: Lists locally available Docker images.

Q16. What does docker run myapp do?
Answer: Creates and starts a container from the myapp image.

Q17. What is the expected output?
Answer: Hello from Docker!

Q18. Image vs container?
Answer: An image is a template; a container is an instance created from it.

Q19. Why use Docker?
Answer: It improves environment consistency, simplifies deployment and reduces dependency problems.

Q20. What is Docker Desktop?
Answer: A desktop application providing tools to run and manage Docker containers.

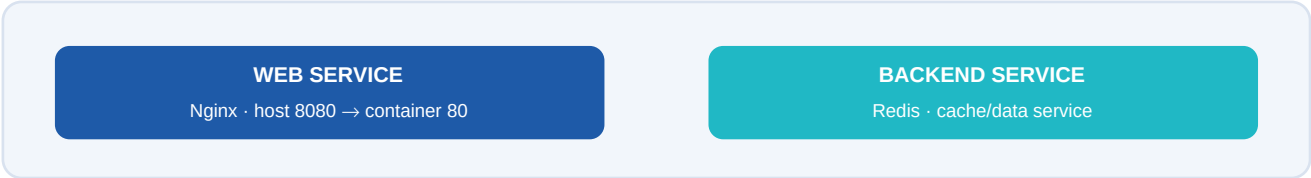
Q21. What is the result of Experiment 4?
Answer: The Python app was built into a Docker image and executed inside a container.

EXPERIMENT 05

Multi-Service Application using Docker Compose

Source PDF pages Pages 27–34 • 19 viva questions

Multi-service architecture



Q01. What is Docker Compose?
Answer: A tool to define and run multi-service applications using a YAML configuration file.

Q02. What is a multi-service application?
Answer: An application made of multiple services that work together.

Q03. Which services are used?
Answer: A web service using Nginx and a Redis service for backend data/cache functionality.

Q04. What is Nginx?
Answer: A web server that serves webpages and handles HTTP requests.

Q05. What is Redis?
Answer: An in-memory data store commonly used for caching and other data operations.

Q06. Which files are created?
Answer: index.html, Dockerfile and docker-compose.yml; the Dockerfile and HTML file are in the app folder.

Q07. What is index.html for?
Answer: It contains the webpage content served by Nginx.

Q08. What does FROM nginx:latest mean?
Answer: Selects Nginx as the base image for the web service.

Q09. What does the COPY instruction do here?
Answer: Copies index.html to /usr/share/nginx/html/index.html, where Nginx serves the page.

Q10. What does docker-compose.yml define?
Answer: Services, images/build instructions, ports and other configuration.

Q11. What does 8080:80 mean?
Answer: Host port 8080 maps to container port 80.

Q12. How do you access the webpage?
Answer: Open http://localhost:8080 while the services are running.

Q13. What command builds and starts the services?
Answer: docker compose up --build

Q14. What does docker compose ps do?
Answer: Displays the status of Compose services and containers.

Q15. How do you stop the app?
Answer: docker compose down stops and removes the Compose containers and network; named volumes are normally retained.

Q16. What should the webpage display?
Answer: Hello Students! Web service is running successfully. This application is deployed using Docker Compose.

Q17. Why use Compose instead of separate commands?
Answer: It keeps service configuration together and manages related services with one command.

Q18. Do Nginx and Redis do the same job?
Answer: No. Nginx serves the webpage; Redis provides a data store/cache service.

Q19. What is the result of Experiment 5?
Answer: The web and Redis services were configured and deployed together using Docker Compose.

EXPERIMENT 06

Deploying and Scaling Applications using Kubernetes

Source PDF pages Pages 35–43 • 21 viva questions

Deployment and replicas



Q01. What is Kubernetes?
Answer: An open-source container orchestration platform used to deploy, manage and scale containerized applications.

Q02. What is container orchestration?
Answer: Automated management of containers, including deployment, scaling, networking and recovery.

Q03. What is Minikube?
Answer: A tool for running a local Kubernetes cluster for learning, development and testing.

Q04. What is kubectl?
Answer: The command-line tool used to manage a Kubernetes cluster.

Q05. What is a Kubernetes cluster?
Answer: A group of nodes that run containerized applications under Kubernetes management.

Q06. What is a node?
Answer: A machine in the cluster that provides resources and runs Pods.

Q07. What is a Pod?
Answer: The smallest deployable Kubernetes unit; it contains one or more containers sharing certain resources.

Q08. What is a Deployment?
Answer: A Kubernetes resource that manages replicated Pods and helps maintain the desired state.

Q09. What is a replica?
Answer: One instance of an application Pod.

Q10. What does minikube start do?
Answer: Starts the local Minikube cluster.

Q11. What does kubectl get nodes do?
Answer: Lists cluster nodes and their status.

Q12. What is a Kubernetes YAML file?
Answer: A configuration file declaring resources and desired settings, such as a Deployment and replica count.

Q13. What does replicas: 2 mean?
Answer: It asks the Deployment to maintain two Pod replicas.

Q14. What does kubectl apply -f deployment.yaml do?
Answer: Creates or updates resources described in deployment.yaml.

Q15. How do you list Pods?
Answer: kubectl get pods

Q16. How do you scale a Deployment to three replicas?
Answer: kubectl scale deployment myapp --replicas=3; replace myapp with the actual Deployment name.

Q17. What is scaling?
Answer: Increasing or decreasing application instances to match workload needs.

Q18. What is self-healing?
Answer: Kubernetes can replace failed Pods to help maintain the desired number of replicas.

Q19. How do you delete a Deployment?
Answer: kubectl delete deployment myapp; replace myapp with the actual Deployment name.

Q20. Docker vs Kubernetes?
Answer: Docker is commonly used to build and run containers; Kubernetes orchestrates containerized apps across a cluster.

Q21. What is the result of Experiment 6?
Answer: A containerized app was deployed and managed with Kubernetes, and replicas were used to demonstrate scaling.

Last-minute command cheat sheet

Revise the command and its purpose. Replace example names with the actual names used in your project.

TOOL	COMMAND	PURPOSE
Git	`git --version`	Check installed version
Git	`git config --global user.name "Your Name"`	Set commit author name
Git	`git config --global user.email "you@example.com"`	Set commit author email
Git	`git init`	Initialize repository
Git	`git status`	Check file status
Git	`git add .`	Stage changes
Git	`git commit -m "Initial commit"`	Record changes
Git	`git log --oneline`	View commit history
Git	`git branch feature`	Create branch
Git	`git checkout feature`	Switch branch
Git	`git checkout main`	Switch to main
Git	`git merge feature`	Merge branch
GitHub	`git clone <repo-url>`	Clone remote repository
GitHub	`git push origin main`	Push main branch
Docker	`docker --version`	Check Docker
Docker	`docker build -t myapp .`	Build image
Docker	`docker images`	List images
Docker	`docker run myapp`	Run container
Compose	`docker compose up --build`	Build/start services
Compose	`docker compose ps`	Check status
Compose	`docker compose down`	Stop/remove containers
Kubernetes	`minikube start`	Start local cluster
Kubernetes	`kubectl get nodes`	List nodes
Kubernetes	`kubectl apply -f deployment.yaml`	Apply YAML resources
Kubernetes	`kubectl get pods`	List Pods
Kubernetes	`kubectl scale deployment myapp --replicas=3`	Scale to 3 replicas
Kubernetes	`kubectl delete deployment myapp`	Delete Deployment

Final viva checklist

- ■ I can define the objective and main tool for every experiment.
- ■ I can explain Agile, Sprint, backlog and Jira board.
- ■ I can explain Git staging, commits, branches and merging.
- ■ I can explain the GitHub Actions trigger and success check.
- ■ I can explain Dockerfile instructions, image vs container and output.
- ■ I can explain Nginx, Redis, port mapping 8080:80 and localhost:8080.
- ■ I can explain Kubernetes Pod, Deployment, replicas and scaling.
- ■ I will use the actual resource names from my setup in a demo.

Viva tip: Be accurate about what you executed. If a command did not run on your machine, explain its purpose without claiming it succeeded.